Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Contents

- Different Image segmentation methods
- Pixel-Based Method Thresholding
- Adaptive Thresholding
- Threshold Level Adjustment
- Thresholding Method
- Clustering based segmentation
- Hierarchical clustering
- Agglomerative clustering
- Mean shift segmentation

Basic Idea of Image Segmentation

- Segmentation is often considered to be the first step in image analysis.
  - The purpose is to subdivide an image into meaningful non-overlapping regions, which would be used for further analysis.
  - It is hoped that the regions obtained correspond to the physical parts or objects of a scene (3-D) represented by the image (2-D).
- In general, autonomous segmentation is one of the most difficult tasks in digital image processing.

Example

Above are two screenshots: a noisy image taken with a mobile phone (converted to greyscale), and the segmented using the threshold selection algorithm. Finally, data collected via border tracing (which requires a binary image) can be used to count the number of fingers in the image.

Pixel-Based Methods

- The most straightforward and common of the pixel-based methods is “thresholding,” in which all pixels having intensity values above, or below, some level are classified as part of the segment.
- Thresholding is an integral part of converting an intensity image to a binary image.
- The basic concept of thresholding can be extended to include both upper and lower boundaries, an operation termed “slicing” since it isolates a specific range of pixels.
- Slicing can be generalized to include a number of different upper and lower boundaries, each encoded into a different number.

Threshold Level Adjustment

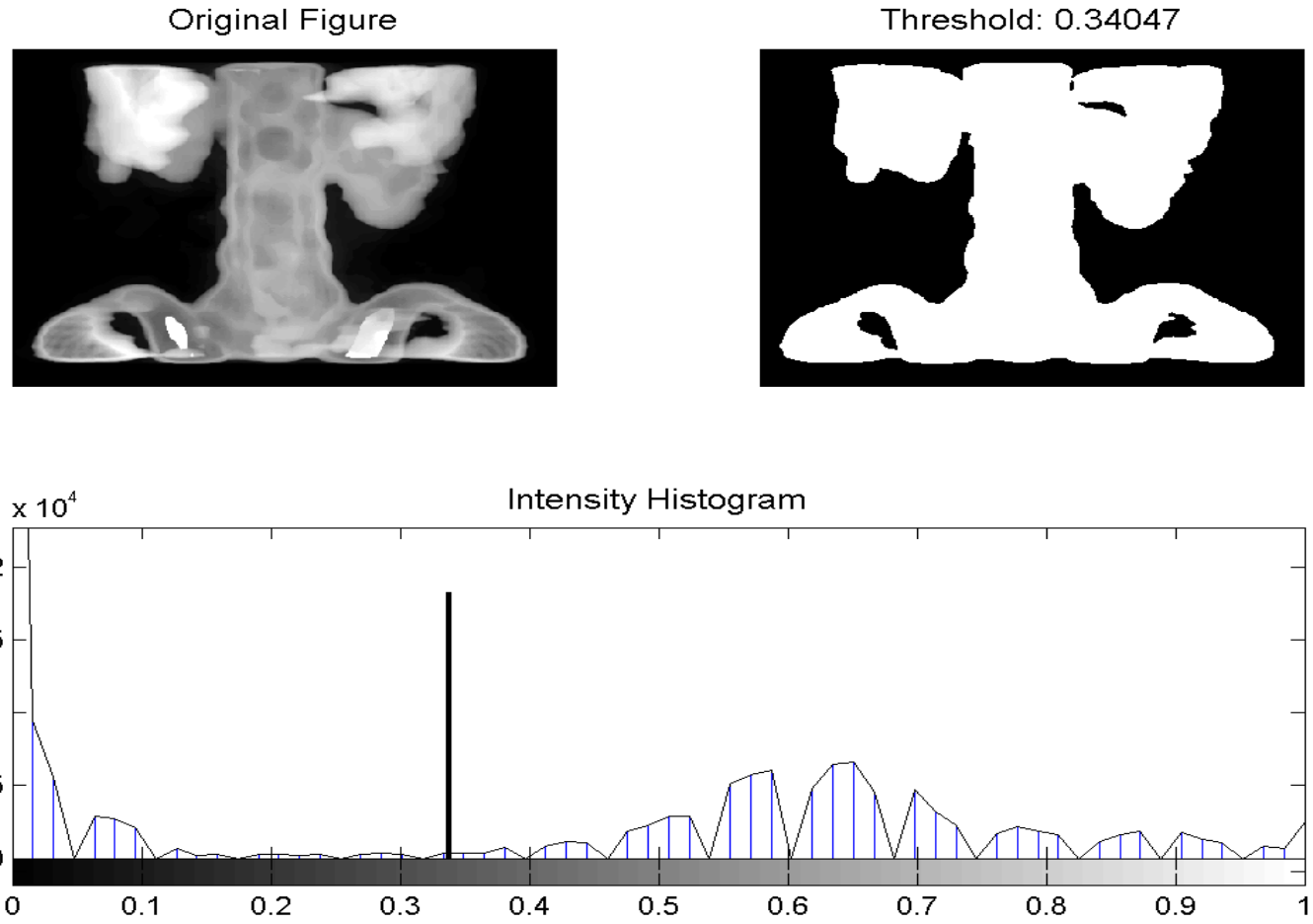
- A major concern in these pixel-based methods is setting the threshold or slicing level(s) appropriately. Usually these levels are set by the program, although in some situations they can be set interactively by the user.

- Finding an appropriate threshold level can be aided by a plot of pixel intensity distribution over the whole image, regardless of whether you adjust pixel level interactively or automatically. Such a plot is termed the “intensity histogram”.

Intensity Histogram

- The intensity histogram is a plot of the number of pixels having a given intensity against intensity.
- If the intensity histogram is, or can be assumed to be, bimodal (or multi-modal), a common strategy is to search for low points, or minima, in the histogram.
- Such points represent the fewest number of pixels and should produce minimal classification errors; however, the histogram minima are often difficult to determine due to variability.

Intensity Histogram

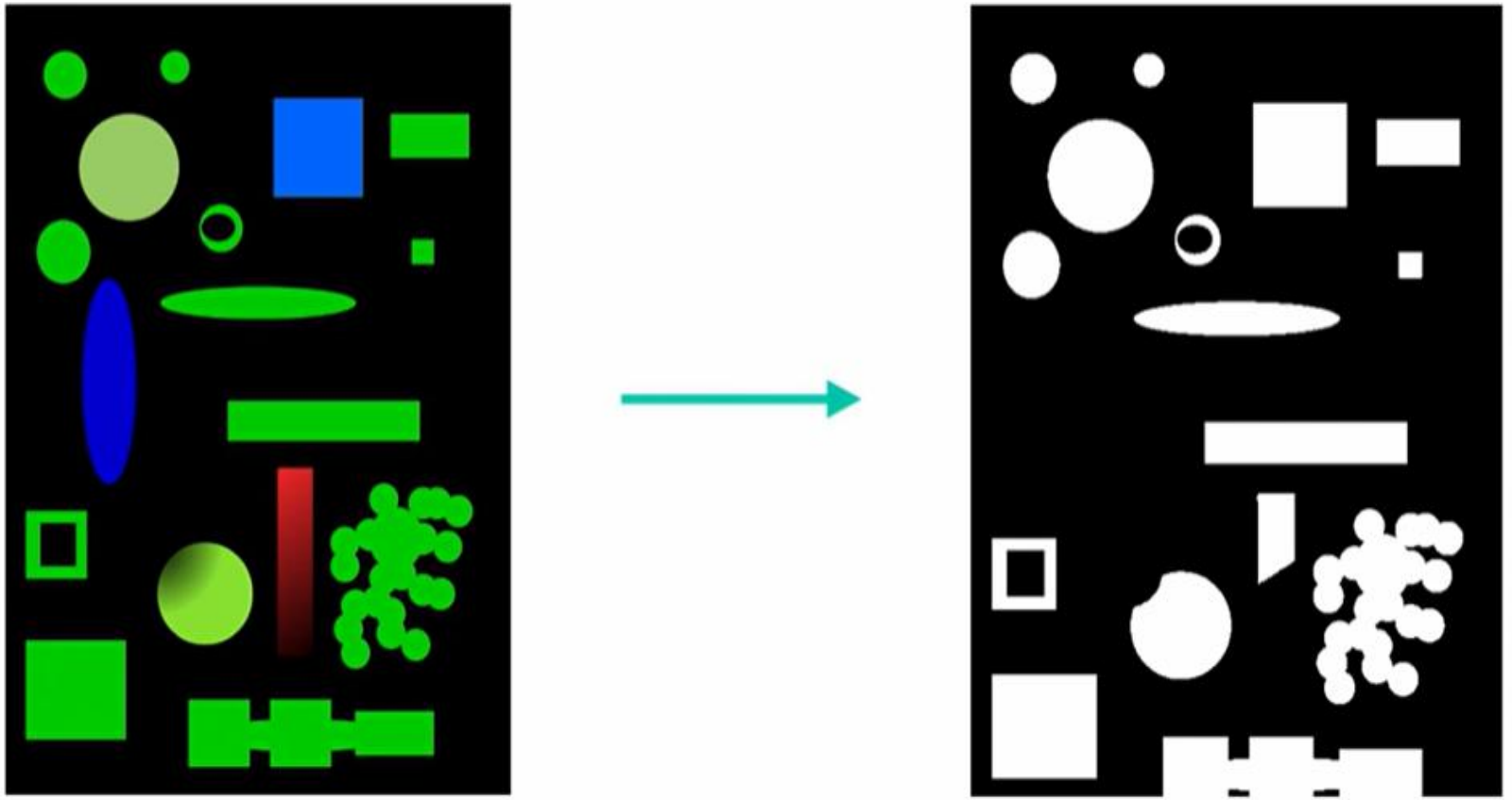


- The intensity histogram of an x-ray image of the spine (lower trace). Setting a threshold between the two peaks separates the image from the background.

Intensity Histogram

- An approach to improve the determination of histogram minima is based on the observation that many boundary points carry values intermediate to the values on either side of the boundary.
- These intermediate values will be represented in the region between the actual boundary values and may mask the optimal threshold value.
- These intermediate points will also have the highest gradient, and it should be possible to identify them using a gradient-sensitive filter such as the Sobel or Canny filter.
- After these boundary points are identified, they can be eliminated from the image and a new histogram computed with distribution that is possibly more definitive.

Segmentation



Thresholding Method

- Basic Idea:

  Select a threshold.

  Pixels above threshold -> region 1

  Pixels below threshold -> region 2

Main challenge -> selection of threshold

Thresholding Method

- Advantages:
  - Simplicity of Implementation
  - Useful if histogram has well separated and distinct peaks

- Disadvantages:
  - Poor noise performance.
  - Sensitive to intensity inhomogeneities

Binary image

0	64	190	255
0	64	190	255
0	64	190	255
0	64	190	255
0	64	190	255

Binary threshold

$ave(p_{xy}) \geq 128$

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

0	64	190	255
0	64	190	255
0	64	190	255
0	64	190	255
0	64	190	255

Binary threshold

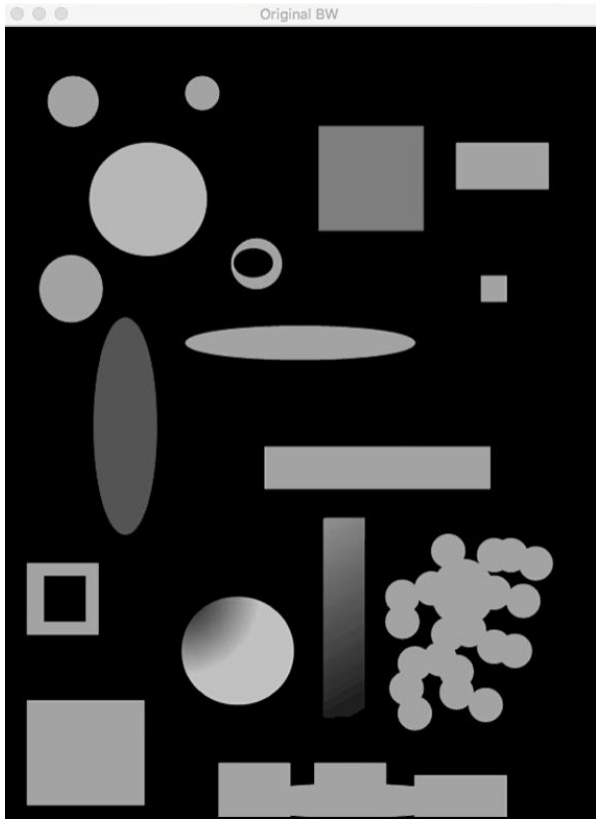
$ave(p_{xy}) \geq 64$

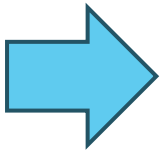
0	1	1	1
0	1	1	1
0	1	1	1
0	1	1	1
0	1	1	1

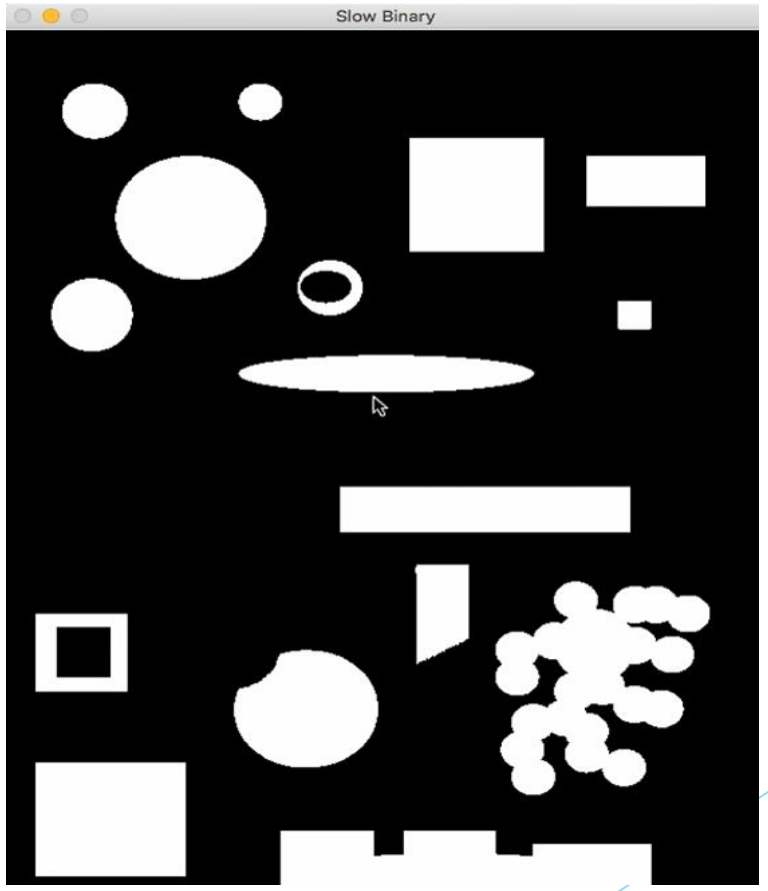
Thresholding practice code

```
import numpy as np
import cv2

bw = cv2.imread('detect_blob.png', 0)
height, width = bw.shape[0:2]
cv2.imshow("Original BW",bw)

binary = np.zeros([height,width,1],'uint8'

thresh = 85

for row in range(0,height):
    for col in range(0, width):
        if bw[row][col]>thresh:
            binary[row][col]=255

cv2.imshow("Slow Binary",binary)


cv2.waitKey(0)
cv2.destroyAllWindows()
```

Simple Thresholding



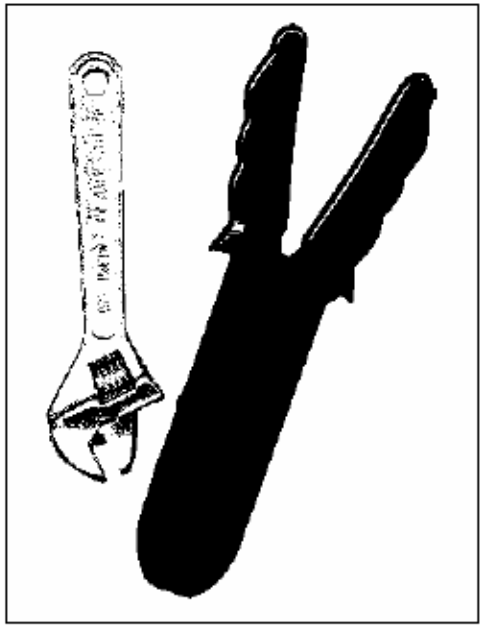




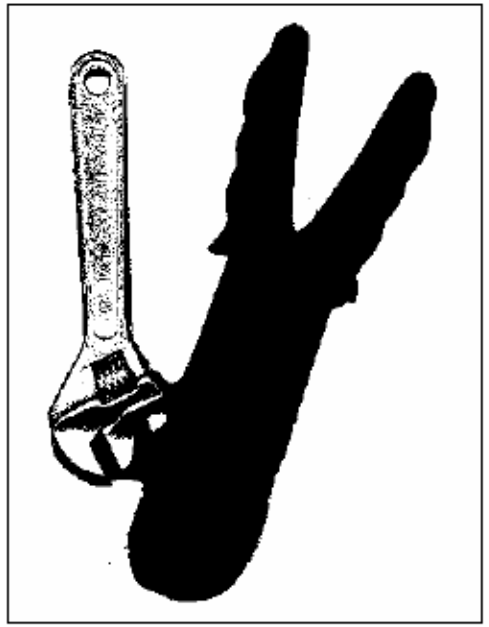
Basic Idea of Image Segmentation

- All the image segmentation methods assume that:
  1. the intensity values are different in different regions, and,
  2. within each region, which represents the corresponding object in a scene, the intensity values are similar.

Original



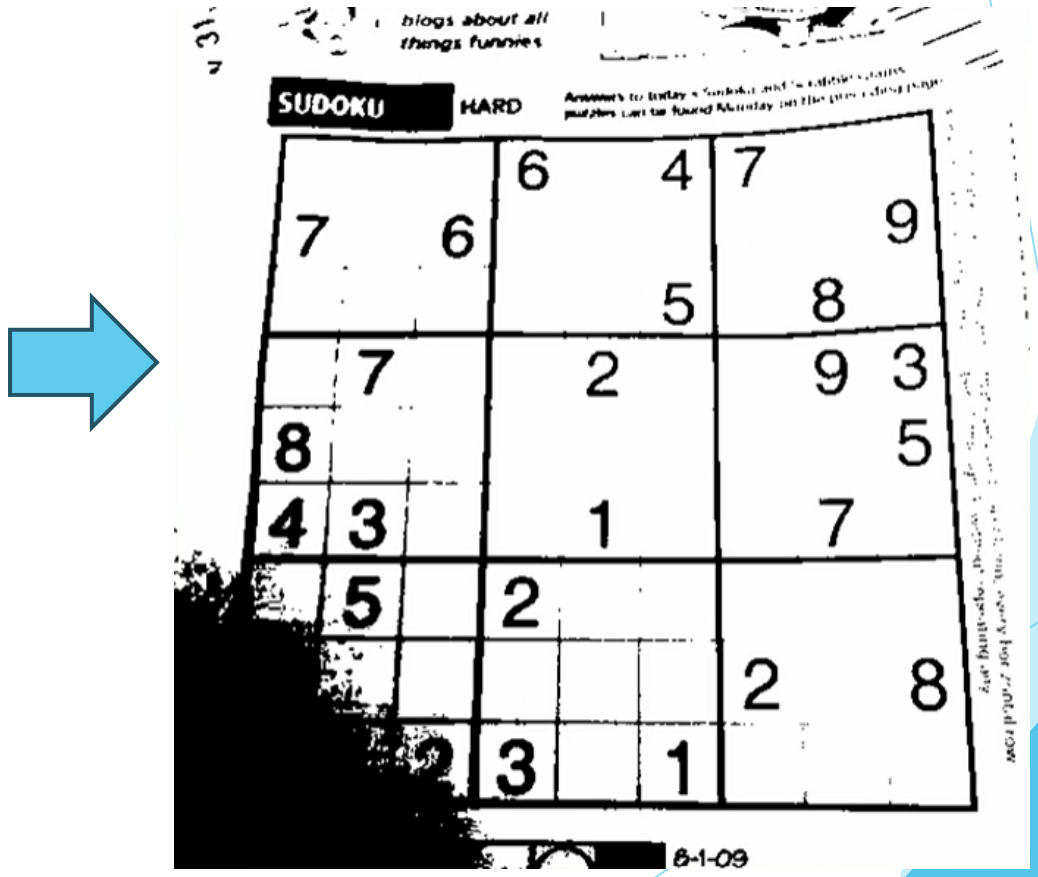
Threshold = 50



Threshold = 75

```
im=imread('tools.bmp');
» imshow(im)
» figure(2); im1=imshow(double(im)>50);
» figure(3); im1=imshow(double(im)>80);
```

Basic thresholding



Adaptive thresholding

if an image has different lighting conditions in different areas. In that case, adaptive thresholding can help.

```
import numpy as np
import cv2

img = cv2.imread('sudoku.png',0)
cv2.imshow("Original",img)

ret, thresh_basic = cv2.threshold(img,70,255,cv2.THRESH_BINARY)
cv2.imshow("Basic Binary",thresh_basic)

thres_adapt = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 1
cv2.imshow("Adaptive Threshold",thres_adapt)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Adaptive thresholding

Images having uneven illumination makes it difficult to segment using histogram, this approach is to divide the original image into sub images and use the thresholding process to each of the sub images.

Fixed Thresholding

- In fixed (or global) thresholding, the threshold value is held constant throughout the image:
- Determine a single threshold value by treating each pixel independently of its neighborhood.

- Fixed thresholding is of the form:

$$g(x,y) = \begin{cases} 0 & f(x,y)<T \\ 1 & f(x,y)>=T \end{cases}$$

  - Assumes high-intensity pixels are of interest, and low-intensity pixels are not.

Threshold Values & Histograms

- Thresholding usually involves analyzing the histogram
  - Different features give rise to distinct features in a histogram
  - In general the histogram peaks corresponding to two features will overlap. The degree of overlap depends on peak separation and peak width.
- An example of a threshold value is the mean intensity value

Automatic Thresholding Algorithm

- Select an initial estimate of the threshold T. A good initial value is the average intensity of the image.
- Calculate the mean grey values μ_1 and μ_2 of the partitions, R1, R2 .
- Partition the image into two groups, R1, R2 , using the threshold T.
- Select a new threshold:

$$T = \frac{1}{2}(\mu_1 + \mu_2)$$

- Repeat steps 2-4 until the mean values μ_1 and μ_2 in successive iterations do not change.

Matlab Implementation

```
>> im =imread('boy.jpg');
>> I=rgb2gray(im);
>> I=double(I);
>> T=opthr(I);
bim=(I>T);
subplot(1,2,1), imshow(I, gray(256));
subplot(1,2,2), imshow(bim)
```

Possible Variations

- Pick an initial threshold value, t (e.g. 128).
- Calculate the mean values in the histogram below (m1) and above (m2) the threshold t.
- Calculate new threshold. tnew = (m1 + m2) / 2.
- If the threshold has stabilized (t = tnew), this is threshold level. Otherwise, t become tnew and reiterate from step 2.

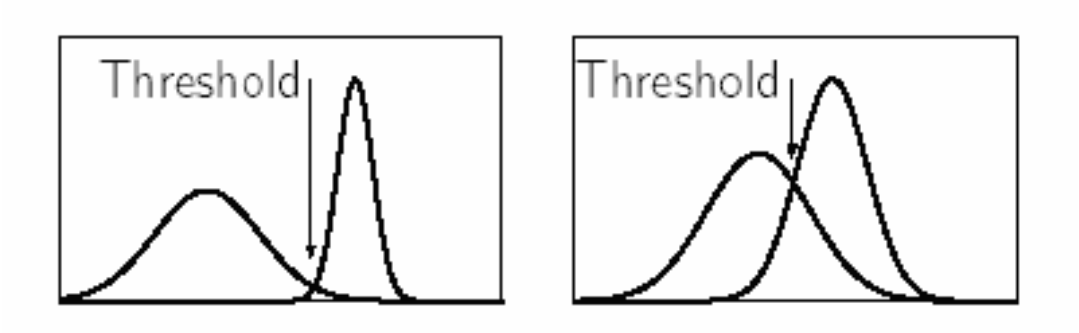
Optimal Thresholding

- Histogram shape can be useful in locating the threshold.

– However it is not reliable for threshold selection when peaks are not clearly resolved.

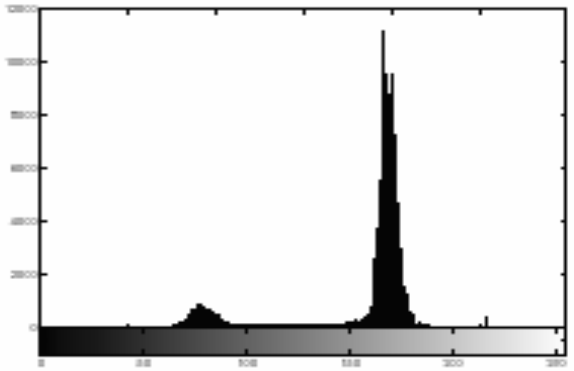
– A “flat” object with no discernable surface texture, and no colour variation will give rise to a relatively narrow histogram peak.

Choosing a threshold in the valley between two overlapping peaks, and inevitably some pixels will be incorrectly classified by the thresholding.

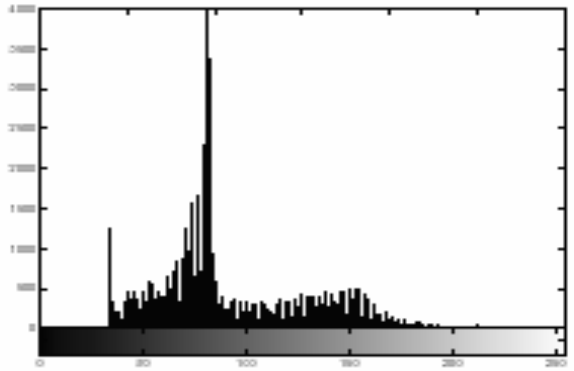


The determination of peaks and valleys is a non-trivial problem

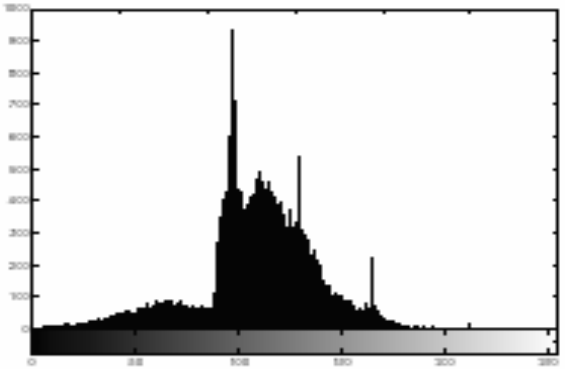
Optimal Thresholding



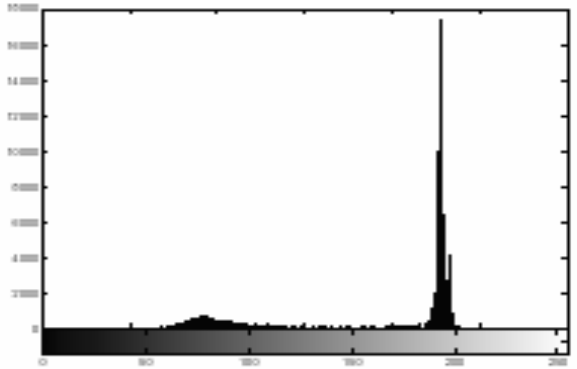
Nodules image: `nodules.tif`



Rice image: `rice.tif`



Bacteria image: `bacteria.tif`



Coins image: `eight.tif`

Optimal Thresholding

- In optimal thresholding, a criterion function is devised that yields some measure of separation between regions.
  - A criterion function is calculated for each intensity and that which maximizes this function is chosen as the threshold.

- Otsu's thresholding chooses the threshold to minimize the intraclass variance of the thresholded black and white pixels or to separate pixels in an image into two classes: foreground and background
  - Formulated as discriminant analysis: a particular criterion function is used as a measure of statistical separation.

Otsu's Methold

- Otsu's thresholding method is based on selecting the lowest point *between* two *classes* (peaks).
- Frequency and Mean value:
  - Frequency: $\omega = \sum_{i=0}^{T} P(i) \qquad P(i) = n_i / N$ N: total pixel number
  - Mean: $\mu = \sum_{i=0}^{T} iP(i) / \omega$ **n_i:** number of pixels in level i
- Analysis of variance (variance=standard deviation^2)
  - Total variance: $\partial_t^2 = \sum_{i=0}^{T} (i-\mu)^2 P(i)$

Otsu's Method

- *between-classes* variance (δ_b^2):

  The variation of the mean values for each class from the overall intensity mean of all pixels:

  $$\delta_b^2 = \omega_0 (\mu_0 - \mu_t)^2 + \omega_1 (\mu_1 - \mu_t)^2,$$

  Substituting $\mu_t = \omega_0 \mu_0 + \omega_1 \mu_1$, we get:

  $$\delta_b^2 = \omega_0 \omega_1 (\mu_1 - \mu_0)^2$$

  ω_0, ω_1, μ_0, μ_1 stands for the frequencies and mean values of two classes, respectively.

Otsu's Method

- The criterion function involves *between-classes* variance to the total variance is defined as:

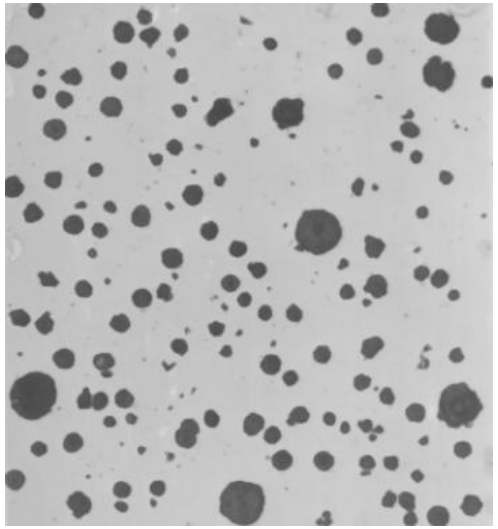
$$\eta = \delta_b^2 / \delta_t^2$$

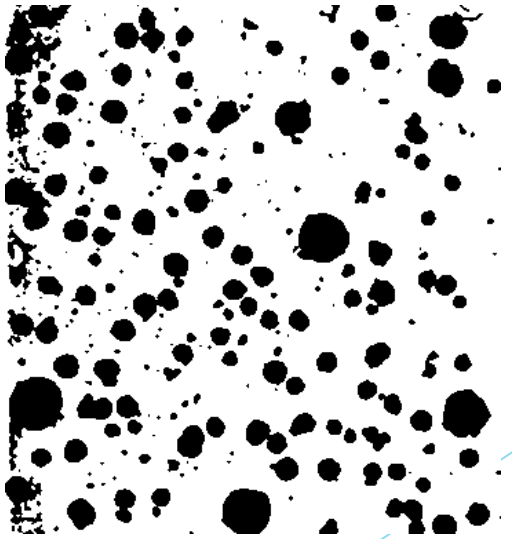
- All possible thresholds are evaluated in this way, and the one that maximizes η is chosen as the optimal threshold

Matlab function for Otsu's method

function level = graythresh(I)
GRAYTHRESH Compute global threshold using Otsu's method. Level is a normalized intensity value that lies in the range [0, 1].

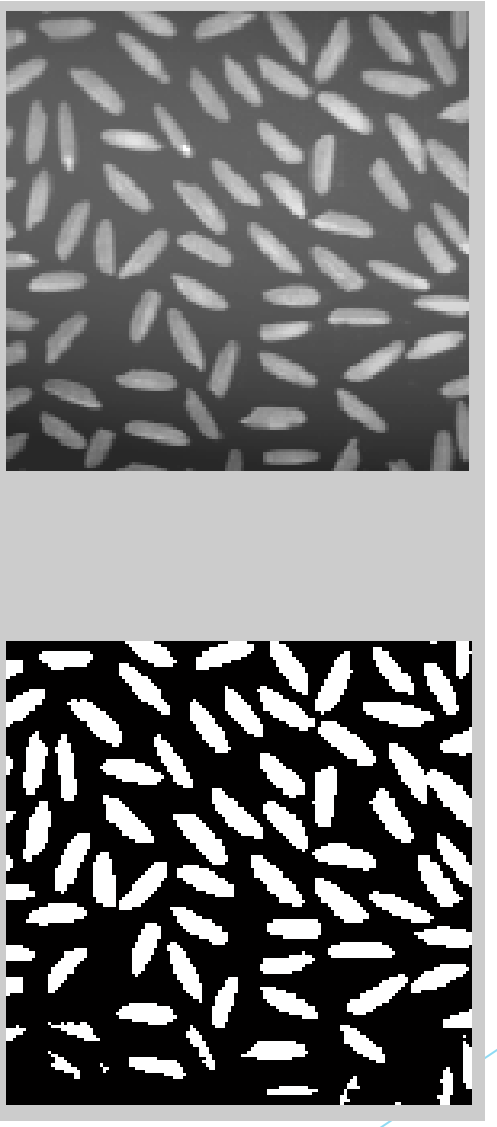
```
>>n=imread('nodules1.tif');
>> tn=graythresh(n)
tn = 0.5804
>> imshow(im2bw(n,tn))
```





Matlab function for Otsu's method

```
r=imread('rice.tif');
graythresh;
subplot(211);
imshow;
subplot(212);
imshow(im2bw(r,tr))
```



Thresholding Bimodal Histograms

- Basic Global Thresholding:

1) Select an initial estimate for T

2) Segment the image using T. This will produce two groups of pixels. G1 consisting of all pixels with gray level values >T and G2 consisting of pixels with values <=T.

3) Compute the average gray level values mean1 and mean2 for the pixels in regions G1 and G2.

4) Compute a new threshold value

$$T=(1/2)(mean1 + mean2)$$

5) Repeat steps 2 through 4 until difference in T in successive iterations is smaller than a predefined parameter T0.

Gray Scale Image - bimodal

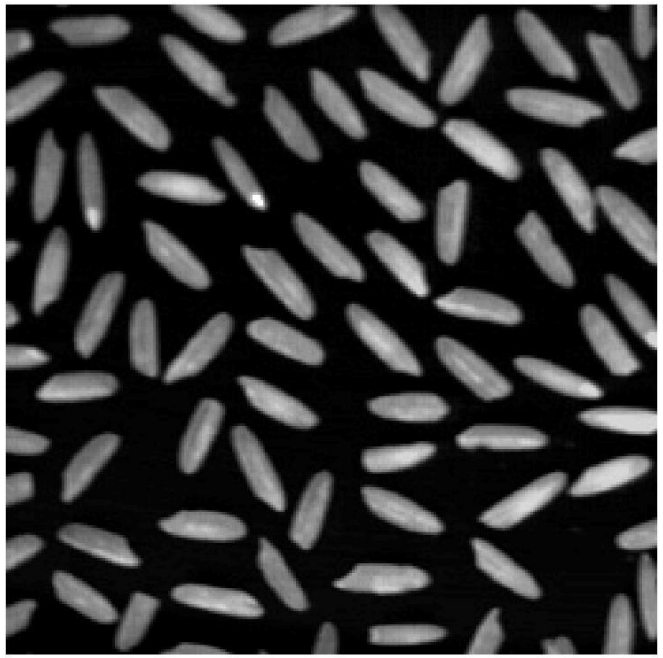


Image of rice with black background

Multimodal Histogram

- If there are three or more dominant modes in the image histogram, the histogram has to be partitioned by multiple thresholds.

- Multilevel thresholding classifies a point (x,y) as belonging to one object class

  if T1 < (x,y) <= T2,

  to the other object class

  if f(x,y) > T2

  and to the background

  if f(x,y) <= T1.

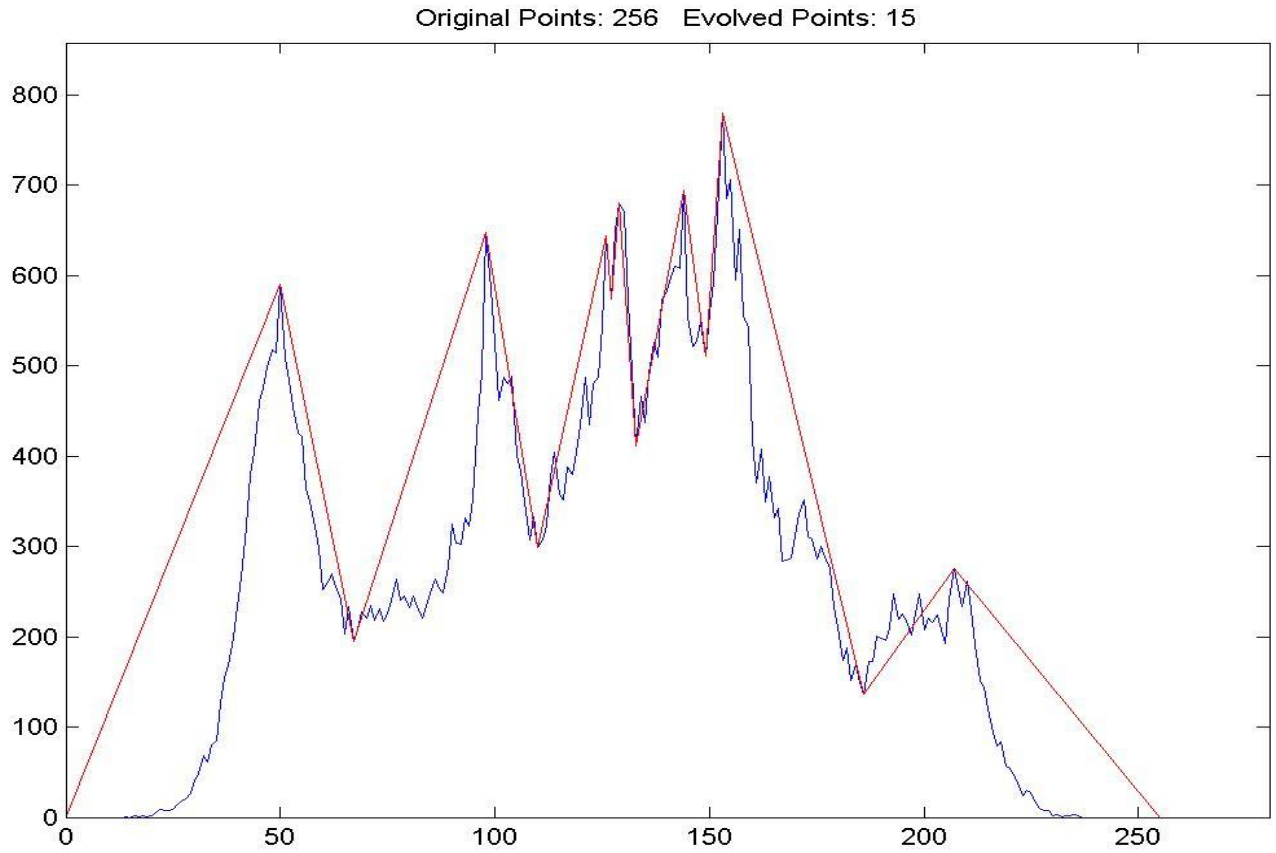
Thresholding multimodal histograms

- A method based on Discrete Curve Evolution to find thresholds in the histogram.
- The histogram is treated as a polyline and is simplified until a few vertices remain.
- Thresholds are determined by vertices that are local minima.

Gray Scale Image - Multimodal

Original Image of lena

Multimodal Histogram



Histogram of lena

Segmented Image

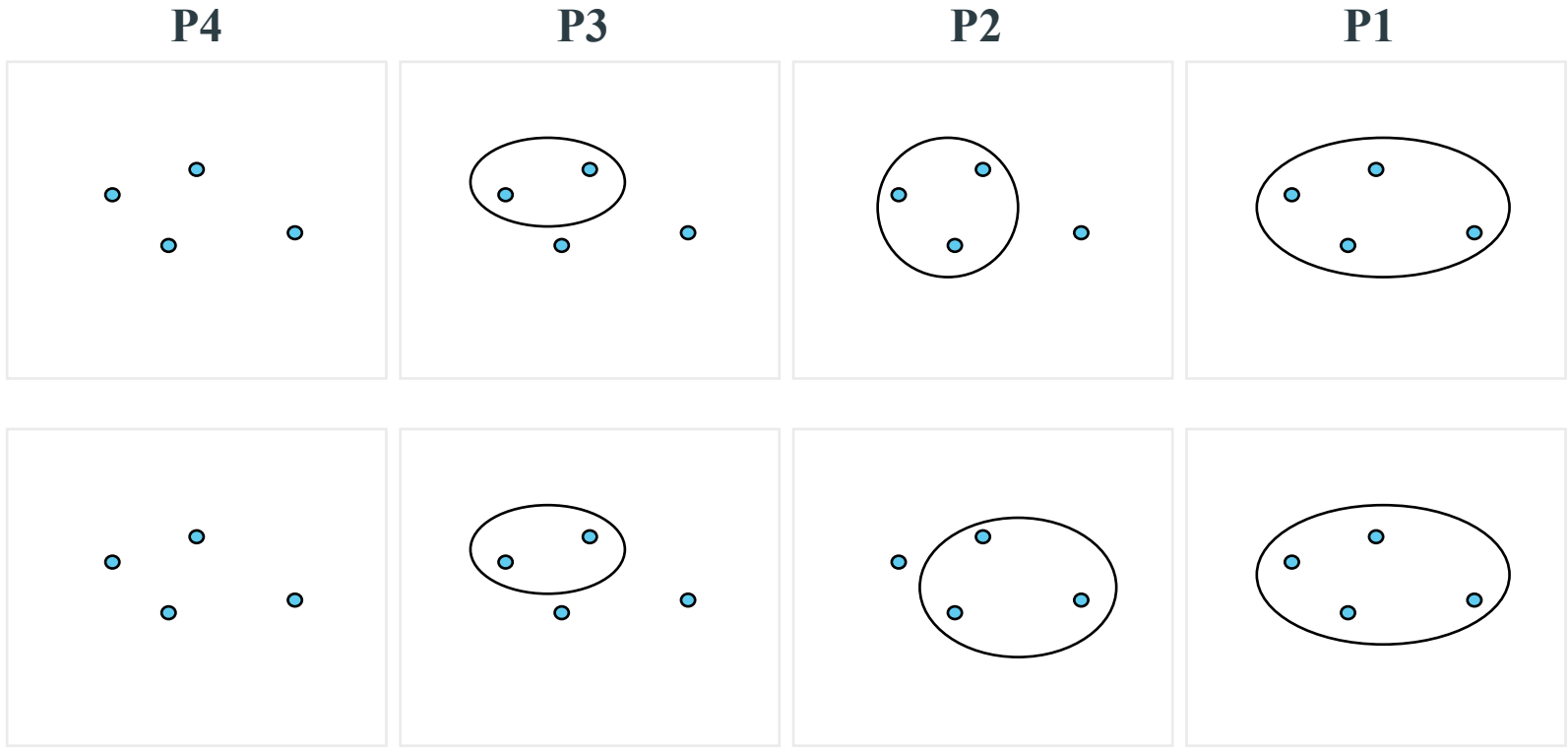
Image after segmentation

Hierarchical clustering

- Flat methods generate a single partition into k clusters. The number k of clusters has to be determined by the user ahead of time.
- Hierarchical methods generate a hierarchy of partitions, i.e.
  - a partition P_1 into 1 clusters (the entire collection)
  - a partition P_2 into 2 clusters
- …
  - a partition P_n into n clusters (each object forms its own cluster)
- It is then up to the user to decide which of the partitions reflects actual sub-populations in the data.

Hierarchical clustering

- A sequence of partitions is called "hierarchical" if each cluster in a given partition is the union of clusters in the next larger partition



Top: hierarchical sequence of partitions
Bottom: non hierarchical sequence

Hierarchical clustering

- Hierarchical methods come in two varieties, **agglomerative** and **divisive.**

Agglomerative methods:

• Start with partition P_n, where each object forms its own cluster.

• Merge the two closest clusters, obtaining P_{n-1}.

• Repeat merge until only one cluster is left.

Divisive methods

• Start with P_1.

• Split the collection into two clusters that are as homogenous (and as different from each other) as possible.

• Apply splitting procedure recursively to the clusters.

Hierarchical clustering

- Agglomerative methods require a rule to decide which clusters to merge. Typically one defines a distance between clusters and then merges the two clusters that are closest.

- Divisive methods require a rule for splitting a cluster.

Hierarchical agglomerative clustering

Need to define a distance d(P,Q) between groups, given a distance measure d(x,y) between observations.

Commonly used distance measures:

1. $d_1(P,Q) = \min d(x,y)$, for x in P, y in Q (single linkage)
2. $d_2(P,Q) = \text{ave } d(x,y)$, for x in P, y in Q (average linkage)
3. $d_3(P,Q) = \max d(x,y)$, for x in P, y in Q (complete linkage)
4. $$d_4(P,Q) = \left\| \bar{x}_P - \bar{x}_Q \right\|$$ (centroid method)
5. $$d_5(P,Q) = 2\frac{|P||Q|}{|P|+|Q|}\left\| \bar{x}_P - \bar{x}_Q \right\|^2$$ (Ward's method)

d_5 is called Ward's distance.

Hierarchical divisive clustering

There are divisive versions of single linkage, average linkage, and Ward's method.

Divisive version of single linkage:

- Compute minimal spanning tree (graph connecting all the objects with smallest total edge length.
- Break longest edge to obtain 2 subtrees, and a corresponding partition of the objects.
- Apply process recursively to the subtrees.

Agglomerative and divisive versions of single linkage give identical results

Divisive version of Ward's method.

Given cluster R.

Need to find split of R into 2 groups P,Q to minimize

$$RSS(P,Q) = \sum_{i \in P} \left\| \mathbf{x}_i - \bar{\mathbf{x}}_P \right\|^2 + \sum_{j \in Q} \left\| \mathbf{x}_j - \bar{\mathbf{x}}_Q \right\|^2$$

or, equivalently, to maximize Ward's distance between P and Q.

Note: No computationally feasible method to find optimal P, Q for large |R|. Have to use approximation.

Dendograms

Result of hierarchical clustering can be represented as binary tree:

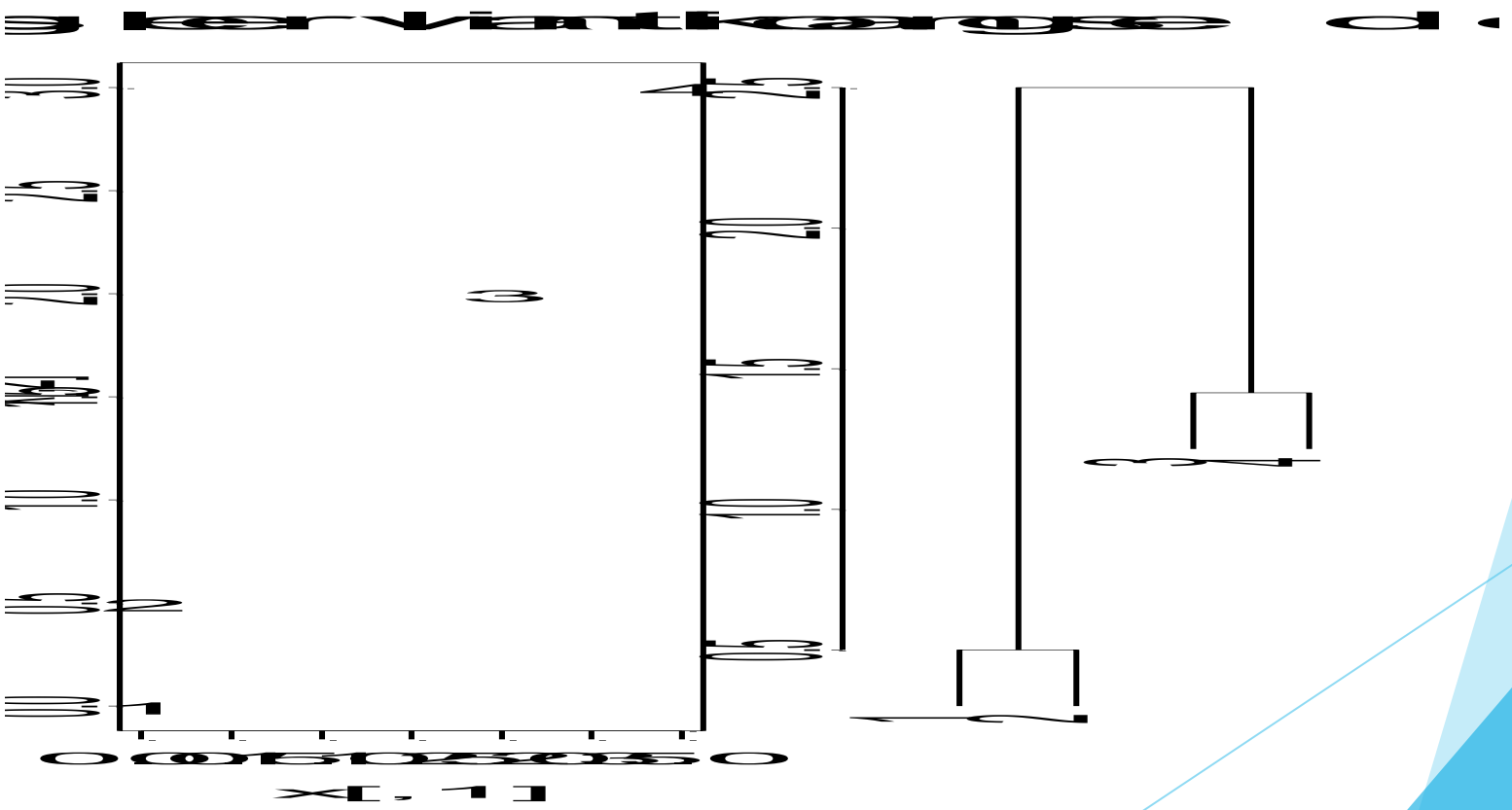
- Root of tree represents entire collection
- Terminal nodes represent observations
- Each interior node represents a cluster
- Each subtree represents a partition

Note: The tree defines many more partitions than the n-2 nontrivial ones constructed during the merge (or split) process.

Dendograms

If distance between daughter clusters is monotonically increasing as we move up the tree, we can draw dendogram:
y-coordinate of vertex = distance between daughter clusters.

Point set and corresponding single linkage dendogram



Standard method to extract clusters from a dendogram:

- Pick number of clusters k.
- Cut dendogram at a level that results in k subtrees.

Agglomerative clustering

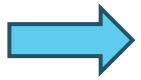
Agglomerative Clustering is a type of hierarchical clustering algorithm. It is an unsupervised machine learning technique that divides the population into several clusters such that data points in the same cluster are more similar and data points in different clusters are dissimilar.

Agglomerative clustering

Consider the six one dimensional data points 18,22, 25,42,27,43

Step 1

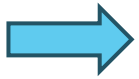
	18	22	25	27	42	43
18	0	4	7	9	24	25
22	4	0	3	5	20	21
25	7	3	0	2	17	18
27	9	5	2	0	15	16
42	24	20	17	15	0	1
43	25	21	18	16	1	0



	18	22	25	27	42	43
18	0	4	7	9	24	25
22	4	0	3	5	20	21
25	7	3	0	2	17	18
27	9	5	2	0	15	16
42	24	20	17	15	0	1
43	25	21	18	16	1	0

Agglomerative clustering

	18	22	25	27	42, 43
18	0	4	7	9	24
22	4	0	3	5	20
25	7	3	0	2	17
27	9	5	2	0	15
42, 43	24	20	17	15	0



	18	22	25, 27	42, 43
18	0	4	7	24
22	4	0	3	20
25, 27	7	3	0	15
42, 43	24	20	15	0

Agglomerative clustering

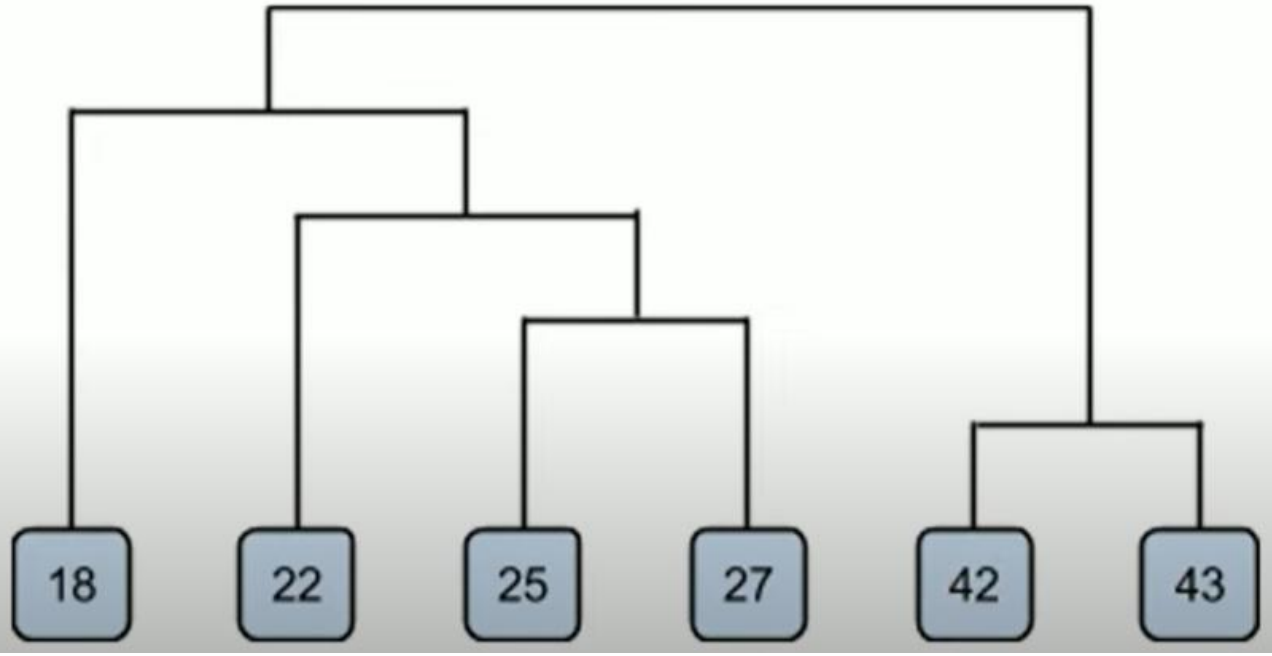
	18, 22, 25, 27	42, 43
18, 22, 25, 27	0	15
42, 43	15	0

	18, 22, 25, 27, 42, 43
18, 22, 25, 27, 42, 43	0

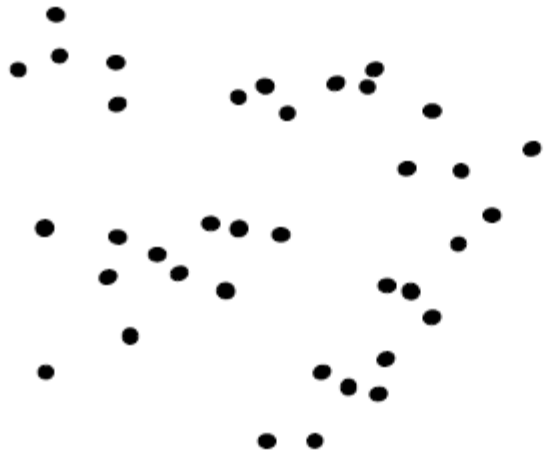
Agglomerative clustering

- **Dendrogram**

((42, 43), (((25, 27), 22), 18))

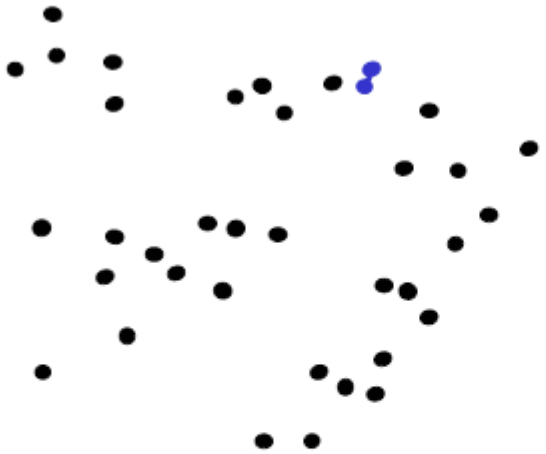


Agglomerative clustering



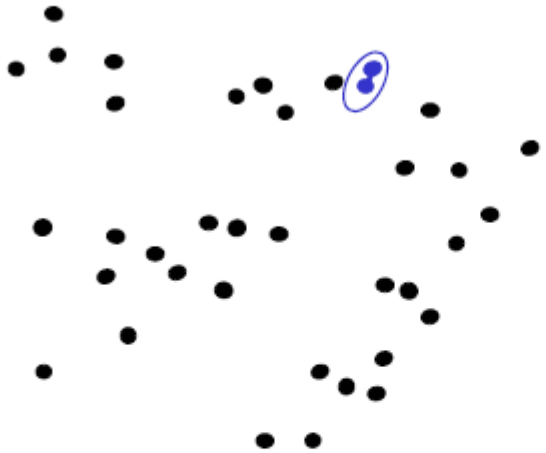
1. Say "Every point is its own cluster"

Agglomerative clustering



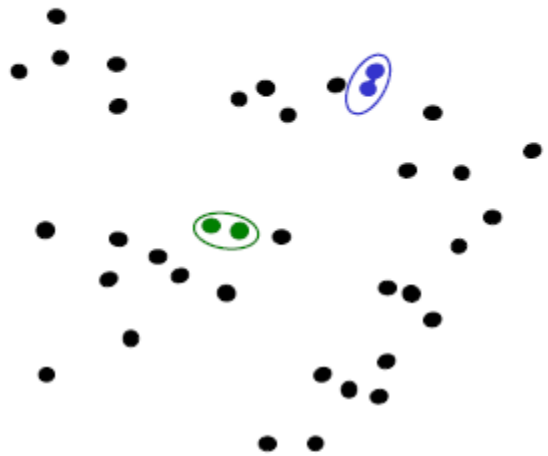
1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters

Agglomerative clustering



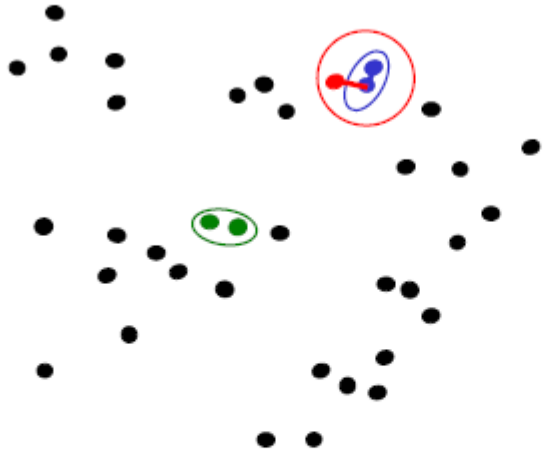
1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster

Agglomerative clustering



1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster
4. Repeat

Agglomerative clustering

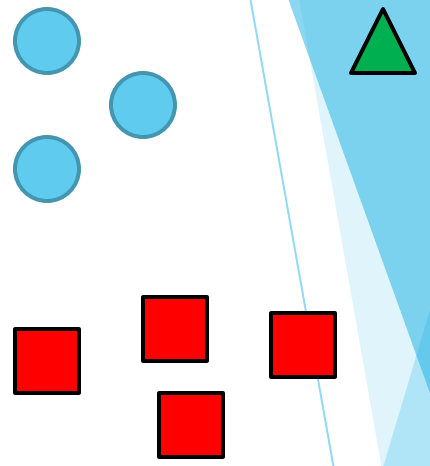


1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster
4. Repeat

Agglomerative clustering

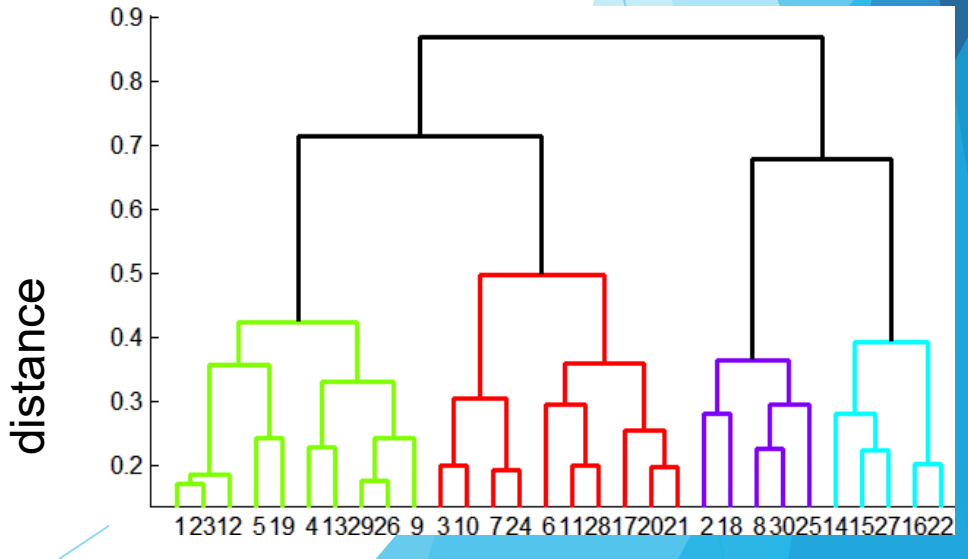
How to define cluster similarity?

- Average distance between points, maximum distance, minimum distance
- Distance between means or medoids



How many clusters?

- Clustering creates a dendrogram (a tree)
- Threshold based on max number of clusters or based on distance between merges



Conclusions: Agglomerative Clustering

Good

- Simple to implement, widespread application
- Clusters have adaptive shapes
- Provides a hierarchy of clusters

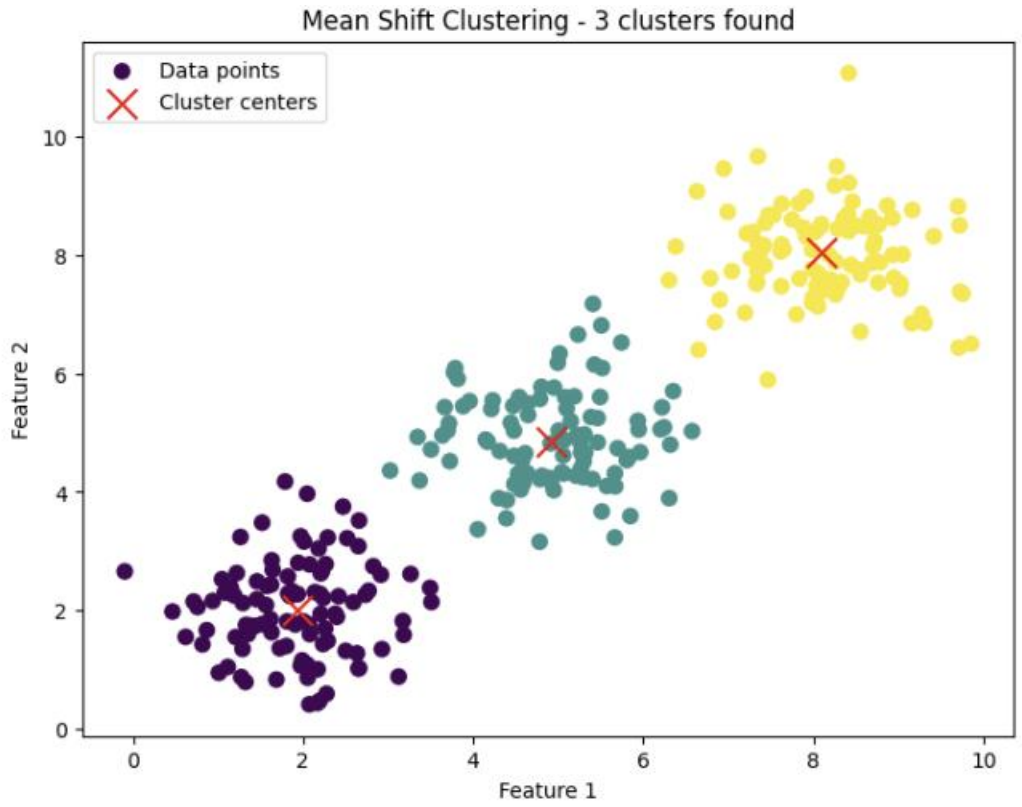
Bad

- May have imbalanced clusters
- Still have to choose number of clusters or threshold
- Need to use an “ultrametric” to get a meaningful hierarchy

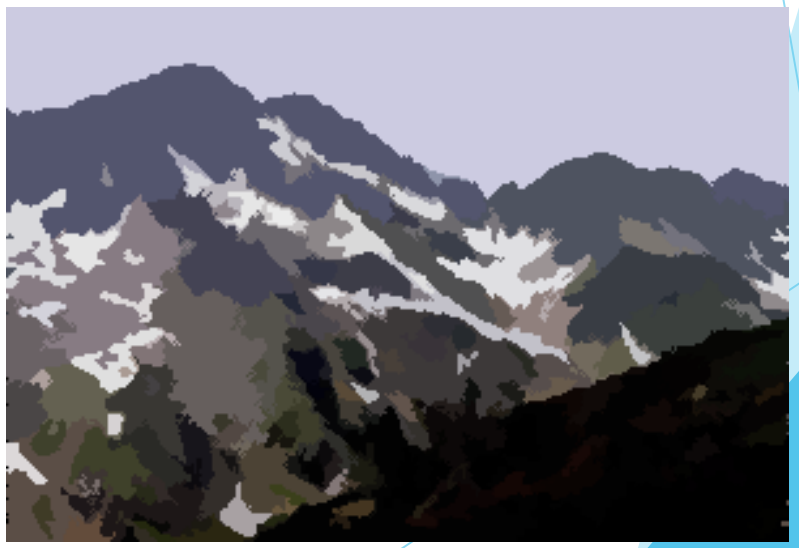
Practice code mean shift

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import MeanShift
from sklearn.datasets import make_blobs

# Generate synthetic data (for example purposes)
centers = [[2, 2], [8, 8], [5, 5]]  # Centers of the clusters
X, _ = make_blobs(n_samples=300, centers=centers, cluster_std=0.8, random_state=42)

# Initialize Mean Shift model
mean_shift = MeanShift(bandwidth=2)  # You can adjust the bandwidth
# Fit the model
mean_shift.fit(X)

# Get the labels and cluster centers
labels = mean_shift.labels_
cluster_centers = mean_shift.cluster_centers_

# Print the number of unique clusters found
n_clusters = len(np.unique(labels))
print(f'Number of clusters found: {n_clusters}')

# Plotting the results
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', marker='o', s=50, label='Data points')
plt.scatter(cluster_centers[:, 0], cluster_centers[:, 1], c='red', marker='x', s=200, label='Cluster centers')

plt.title(f'Mean Shift Clustering - {n_clusters} clusters found')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```

Practice code mean shift



Mean shift segmentation results



http://www.caip.rutgers.edu/~comanici/MSPAMI/msPamiResults.html

Mean shift pros and cons

- Pros
  - Good general-practice segmentation
  - Flexible in number and shape of regions
  - Robust to outliers
- Cons
  - Have to choose kernel size in advance
  - Not suitable for high-dimensional features
- When to use it
  - Over segmentation
  - Multiple segmentations
  - Tracking, clustering, filtering applications

Question

Thank you